University of Minho
School of Engineering

Intelligent Systems Associate Laboratory



Dados e Aprendizagem Automática

Ensemble Learning

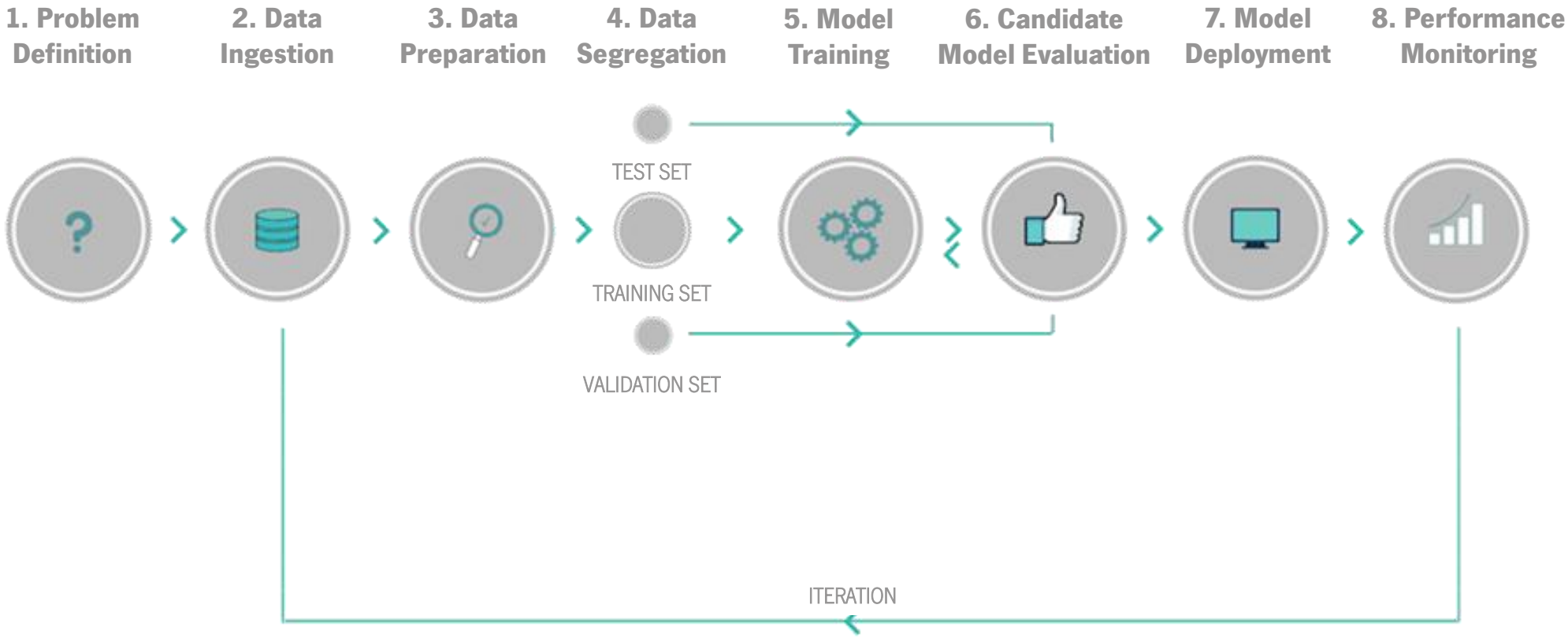
DAA 2026/2027

Filipa Ferraz, Victor Alves, Dalila Durães, Francisco Marcondes, Guilherme Barbosa

Part VI

Contents

- Ensemble Learning
  - Max Voting
  - Bagging
  - Boosting
  - Stacking
- Hands On


1. Problem Definition
2. Data Ingestion
3. Data Preparation
4. Data Segregation
5. Model Training
6. Candidate Model Evaluation
7. Model Deployment
8. Performance Monitoring
TEST SET
TRAINING SET
VALIDATION SET
ITERATION

Ensemble Learning

The Problem and the Data

Problem: development of a Machine Learning model that can **predict which passengers survived** the Titanic disaster.

Dataset: information regarding the **passengers** with 891 entries and 12 features, including:

- *PassengerId*
- *Survived*
- *Pclass*
- *Name*
- *Sex*
- *Age*
- *SibSp*
- *Parch*
- *Ticket*
- *Fare*
- *Cabin*
- *Embarked*

You may need to install `xgboost`. Use one of the following commands:

```
conda install -c conda-forge xgboost

pip install xgboost
```

EDA


```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    object
 4   Sex          891 non-null    object
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    object
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object
 11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

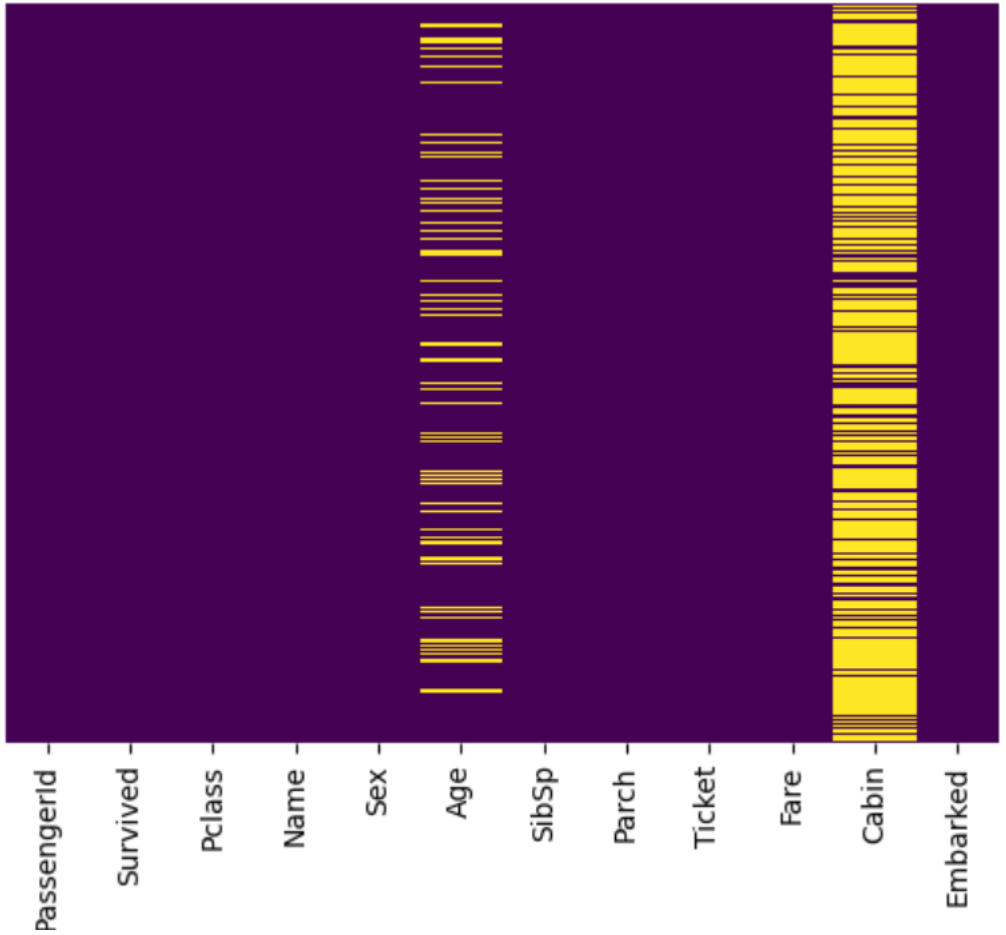
```
df.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S

EDA

We need to check the missing values:

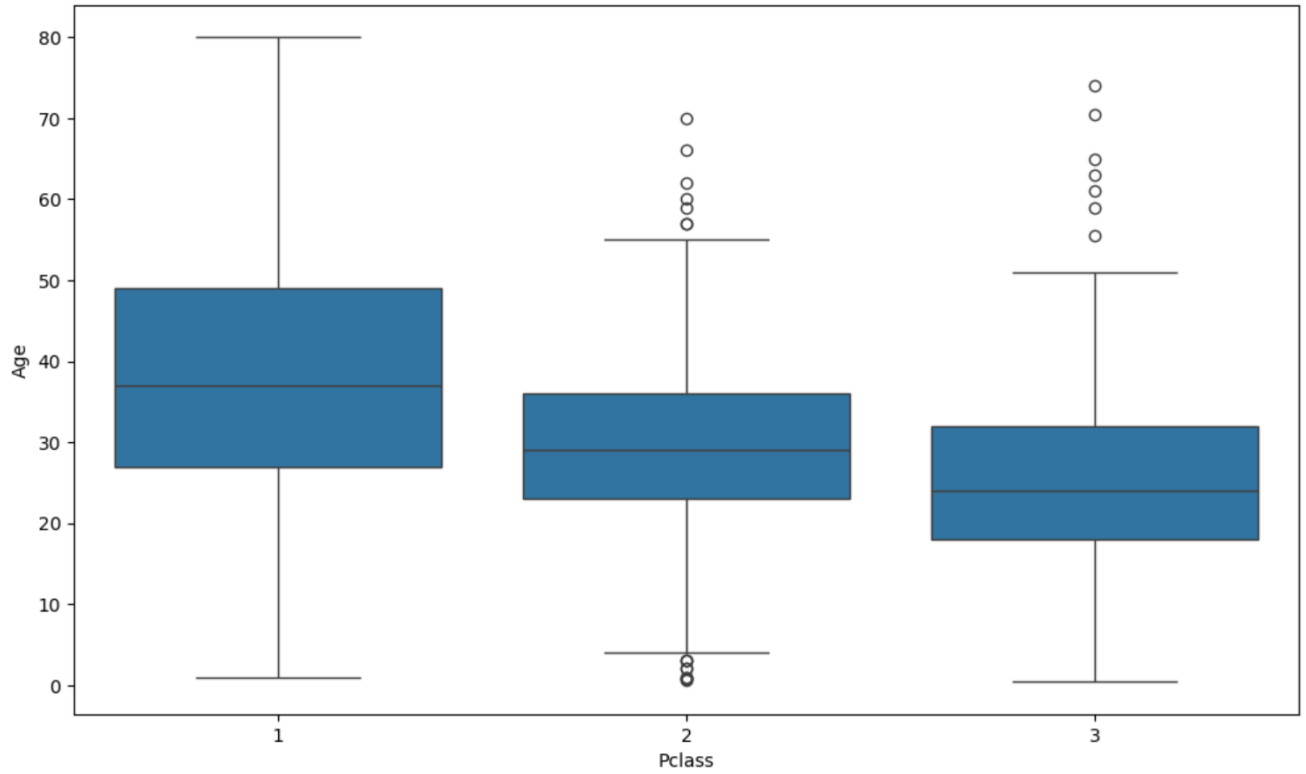
```
sns.heatmap(df.isnull(), yticklabels=False, cbar=False, cmap='viridis')
```



EDA/Data Preprocessing

Analyze the *Age* distribution by ticket class (*Pclass*):

```
plt.figure(figsize = (12, 7))
sns.boxplot(x = 'Pclass', y = 'Age', data = df)
```



We are going to impute missing *Age* values using the ticket class (*Pclass*):

```
df['Age'] = df['Age'].fillna(df.groupby('Pclass')['Age'].transform('median'))
```

Data Preprocessing

For now, we will remove the *Cabin*, *Sex*, *Embarked*, *Name* and *Ticket* features:

```
df.drop('Cabin', axis = 1, inplace = True)
df.dropna(inplace = True)
```

```
df.drop(['Sex', 'Embarked', 'Name', 'Ticket'], axis = 1,inplace = True)
```

```
df.head()
```

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
0	1	0	3	22.0	1	0	7.2500
1	2	1	1	38.0	1	0	71.2833
2	3	1	3	26.0	0	0	7.9250
3	4	1	1	35.0	1	0	53.1000
4	5	0	3	35.0	0	0	8.0500

Define the X and y, and split the data:

```
X = df.drop('Survived', axis = 1)
y = df['Survived']
```

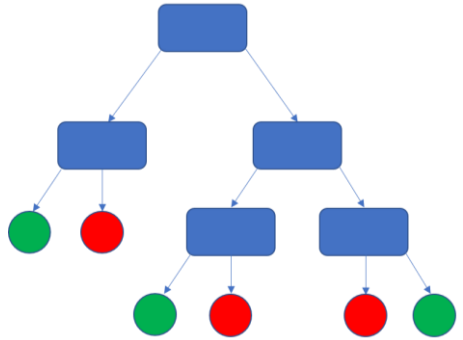
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.30, random_state = 2025)
```

```
X_train.shape, y_train.shape, X_test.shape, y_test.shape
```

```
((622, 6), (622,), (267, 6), (267,))
```

Comparing Models: Decision Tree

We are going try out some different models. We will implement Decision Tree, Logistic Regression and Random Forest models for comparison with Ensemble Learning models.



Decision Tree

```
dt_model = DecisionTreeClassifier(max_depth = 2, random_state = 2025)
```

```
dt_model.fit(X_train, y_train)
```

Evaluate the model and get the predictions:

```
dt_score = dt_model.score(X_test, y_test)
```

```
print("Accuracy: %.2f%%" % (dt_score * 100))
```

```
Accuracy: 68.16%
```

```
dt_predictions = dt_model.predict(X_test)
```

```
print(classification_report(y_test, dt_predictions))
```

	precision	recall	f1-score	support
0	0.72	0.78	0.75	162
1	0.61	0.53	0.57	105
accuracy			0.68	267
macro avg	0.66	0.66	0.66	267
weighted avg	0.68	0.68	0.68	267

Save the model's accuracy for later comparison:

```
results = {'DT': dt_score}
```

Comparing Models: Logistic Regression

Logistic Regression

```
lr_model = LogisticRegression(random_state=2025, solver='liblinear')
```

```
lr_model.fit(X_train, y_train)
```

```
lr_score = lr_model.score(X_test, y_test)
```

```
print("Accuracy: %.2f%%" % (lr_score * 100))
```

```
Accuracy: 67.79%
```

```
lr_predictions = lr_model.predict(X_test)
```

```
print(classification_report(y_test, lr_predictions))
```

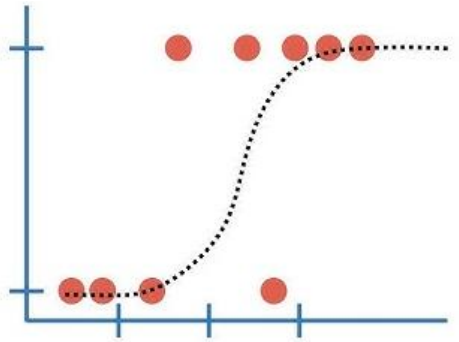
```
              precision    recall  f1-score   support

           0       0.68      0.89      0.77       162
           1       0.67      0.35      0.46       105

    accuracy                           0.68       267
   macro avg       0.68      0.62      0.62       267
weighted avg       0.68      0.68      0.65       267
```

```
results['LogR'] = lr_score
```

```
sklearn.linear_model.LogisticRegression(penalty='l2', *, dual=False, tol=0.0001,
C=1.0, fit_intercept=True, intercept_scaling=1, class_weight=None,
random_state=None, solver='lbfgs', max_iter=100,
multi_class='deprecated', verbose=0, warm_start=False, n_jobs=None,
l1_ratio=None)
```



Comparing Models: Random Forest

Random Forest

```
rf_model = RandomForestClassifier()
```

```
rf_model.fit(X_train, y_train)
```

```
rf_score = rf_model.score(X_test, y_test)
```

```
print("Accuracy: %.2f%%" % (rf_score * 100))
```

```
Accuracy: 71.16%
```

```
rf_predictions = rf_model.predict(X_test)
```

```
print(classification_report(y_test, rf_predictions))
```

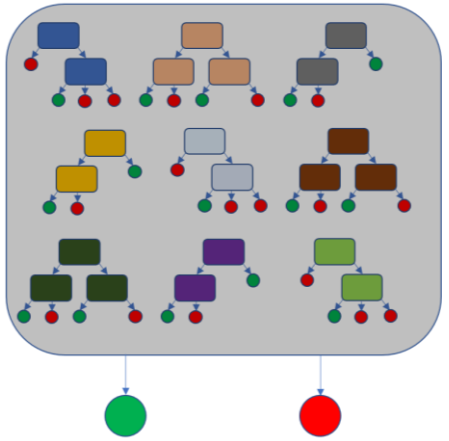
```
              precision    recall  f1-score   support

           0       0.73      0.83      0.78       162
           1       0.67      0.53      0.59       105

    accuracy                           0.71       267
   macro avg       0.70      0.68      0.68       267
weighted avg       0.71      0.71      0.70       267
```

```
results['RF'] = rf_score
```

```
sklearn.ensemble.RandomForestClassifier(n_estimators=100, *, criterion='gini',
                max_depth=None, min_samples_split=2, min_samples_leaf=1,
                min_weight_fraction_leaf=0.0, max_features='sqrt',
                max_leaf_nodes=None, min_impurity_decrease=0.0, bootstrap=True,
                oob_score=False, n_jobs=None, random_state=None, verbose=0,
                warm_start=False, class_weight=None, ccp_alpha=0.0,
                max_samples=None, monotonic_cst=None)
```

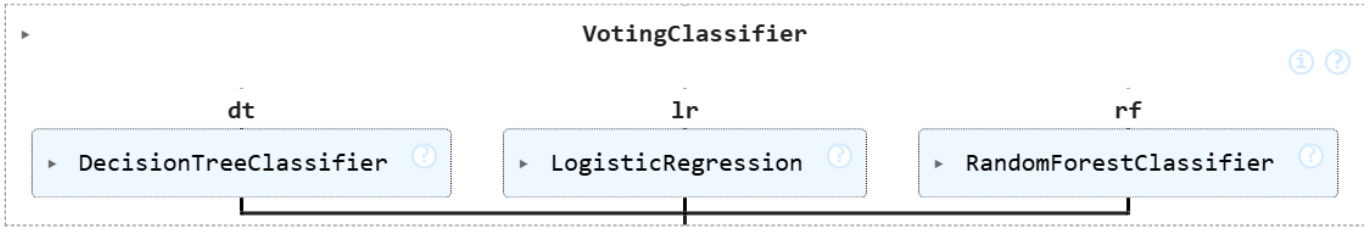


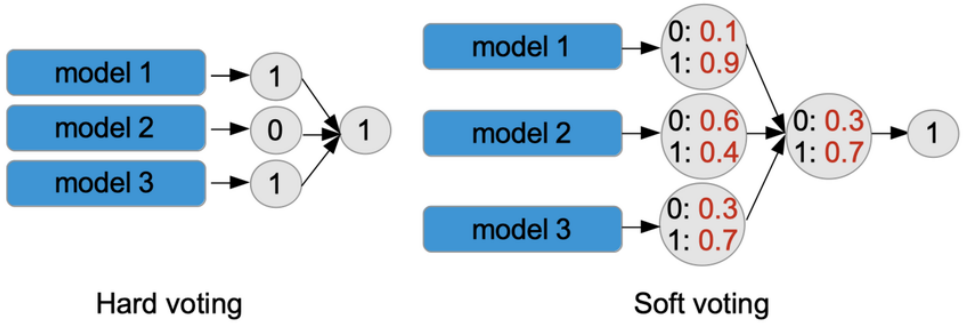
Ensemble Learning Models: Max Voting

Majority Class Labels (Majority/Hard Voting)

```
hvt_model = VotingClassifier(estimators = [("dt", dt_model), ("lr", lr_model), ("rf", rf_model)],
                             voting = 'hard', weights = [2, 1, 2])
```

```
hvt_model.fit(X_train, y_train)
```






```
sklearn.ensemble.VotingClassifier(estimators,
                *, voting='hard', weights=None,
                n_jobs=None,
                flatten_transform=True,
                verbose=False)
```

Max voting involves fitting several models, each making a prediction and voting on each sample. Only the class with the most votes is included in the final prediction.

Ensemble Learning Models: Max Voting


```
for model, label in zip([dt_model, lr_model, rf_model, hvt_model], ['dt', 'lr', 'rf', 'Ensemble']):
    hvt_score = cross_val_score(model, X_test, y_test, scoring = 'accuracy', cv = 5)
    print("Accuracy: %0.2f (+/- %0.2f) [%s]" % (hvt_score.mean(), hvt_score.std(), label))
```

```
Accuracy: 0.60 (+/- 0.04) [dt]
Accuracy: 0.66 (+/- 0.07) [lr]
Accuracy: 0.64 (+/- 0.06) [rf]
Accuracy: 0.65 (+/- 0.08) [Ensemble]
```

```
hvt_score = hvt_model.score(X_test, y_test)
print("Accuracy: %.2f%%" % (hvt_score*100))
```

```
Accuracy: 69.66%
```

```
hvt_predictions = hvt_model.predict(X_test)
```

```
print(classification_report(y_test, hvt_predictions))
```

```
              precision    recall  f1-score   support

           0       0.71      0.84      0.77       162
           1       0.66      0.48      0.55       105

    accuracy                           0.70       267
   macro avg       0.68      0.66      0.66       267
weighted avg       0.69      0.70      0.68       267
```

```
results['HVotW']= hvt_score
```

Ensemble Learning Models: BAGGing *(Bootstrap AGGregating)*

We will use the **StratifiedShuffleSplit** technique which is another form of splitting.

```
sss = StratifiedShuffleSplit(n_splits = 10, test_size = 20, random_state = 2025)
```

```
sklearn.model_selection.StratifiedShuffleSplit(n_splits=10, *,
                test_size=None, train_size=None, random_state=None)
```

Implement the **Bagging Classifier** with Decision Tree:

```
bg_dt_model = BaggingClassifier(estimator = dt_model, bootstrap = True)
```

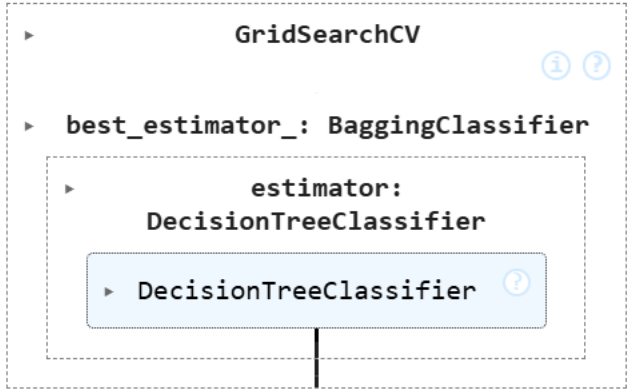
```
n_estimators = [10, 40, 60, 80, 100, 160]
```

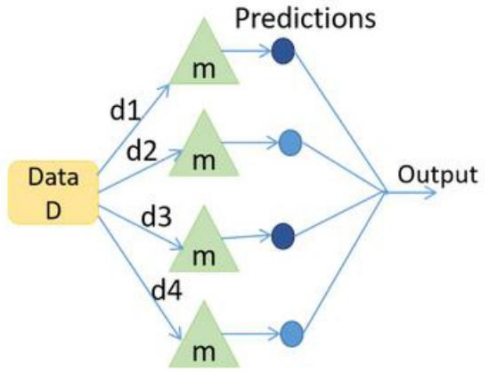
```
parameters = {'n_estimators': n_estimators}
```

```
grid_bg = GridSearchCV(estimator = bg_dt_model, param_grid = parameters, cv = sss)
```

```
grid_bg.fit(X_train, y_train)
```

```
sklearn.ensemble.BaggingClassifier(estimator=None,
                n_estimators=10, *, max_samples=1.0,
                max_features=1.0, bootstrap=True,
                bootstrap_features=False, oob_score=False,
                warm_start=False, n_jobs=None, random_state=None,
                verbose=0)
```





BAGGing involves creating different decision trees from the same dataset and evaluating them by calculating the average of their predictions.

Ensemble Learning Models: Bagging

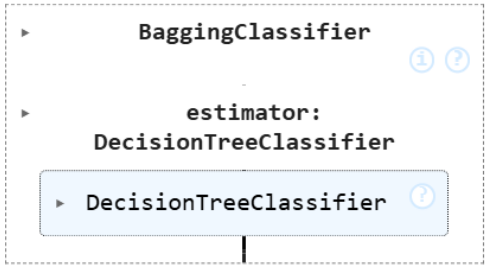
Best estimator

```
bst_bg_model = grid_bg.best_estimator_
```

```
print(bst_bg_model)
```

```
BaggingClassifier(estimator=DecisionTreeClassifier(max_depth=2,
                                                   random_state=2025),
                  n_estimators=80)
```

```
bst_bg_model.fit(X_train, y_train)
```




```
bst_bg_score = bst_bg_model.score(X_test, y_test)
```

```
print("Accuracy: %.2f%%" % (bst_bg_score*100))
```

```
Accuracy: 69.29%
```

```
bg_predictions = bst_bg_model.predict(X_test)
```

```
print(classification_report(y_test, bg_predictions))
```

```
              precision    recall  f1-score   support

           0       0.69      0.88      0.78       162
           1       0.69      0.40      0.51       105

    accuracy                           0.69       267
   macro avg       0.69      0.64      0.64       267
weighted avg       0.69      0.69      0.67       267
```

```
results['BaggDT']= bst_bg_score
```

Ensemble Learning Models: Bagging

Bagging with **Random Forest**:

```
bg_rf_model = BaggingClassifier(estimator = rf_model, bootstrap = True)
```

```
bg_rf_model.fit(X_train, y_train)
```

```
bg_rf_score = bg_rf_model.score(X_test, y_test)
print("Accuracy: %.2f%%" % (rf_score * 100))
```

```
Accuracy: 70.41%
```

```
bg_rf_predictions = bg_rf_model.predict(X_test)
```

```
print(classification_report(y_test, bg_rf_predictions))
```

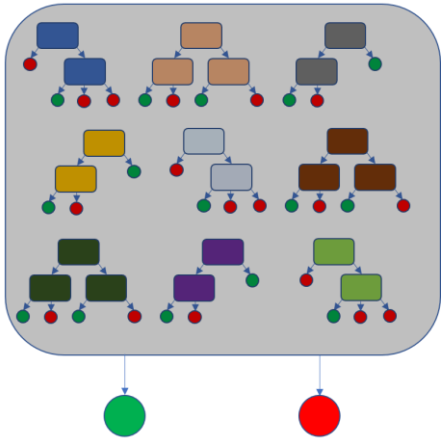
```
              precision    recall  f1-score   support

           0       0.73      0.83      0.77       162
           1       0.66      0.52      0.59       105

    accuracy                           0.71       267
   macro avg       0.70      0.68      0.68       267
weighted avg       0.70      0.71      0.70       267
```

```
results['BaggRF'] = bg_rf_score
```

```
sklearn.ensemble.RandomForestClassifier(n_estimators=100, *,
        criterion='gini', max_depth=None, min_samples_split=2,
        min_samples_leaf=1, min_weight_fraction_leaf=0.0,
        max_features='sqrt', max_leaf_nodes=None,
        min_impurity_decrease=0.0, bootstrap=True, oob_score=False,
        n_jobs=None, random_state=None, verbose=0, warm_start=False,
        class_weight=None, ccp_alpha=0.0, max_samples=None,
        monotonic_cst=None)
```



Ensemble Learning Models: Boosting

Gradient Boosting

```
gbc_model = GradientBoostingClassifier(n_estimators = 100, learning_rate = 1.0,
                                       max_depth = 1, random_state = 2025)
```

```
gbc_model.fit(X_train, y_train)
```

```
gbc_score = gbc_model.score(X_test, y_test)
```

```
print("Accuracy: %.2f%%" % (gbc_score*100))
```

```
Accuracy: 68.54%
```

```
gbc_predictions = gbc_model.predict(X_test)
```

```
print(classification_report(y_test, gbc_predictions))
```

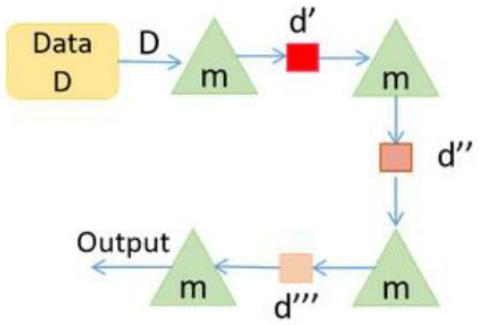
```
              precision    recall  f1-score   support

           0       0.71      0.80      0.76       162
           1       0.62      0.50      0.56       105

    accuracy                           0.69       267
   macro avg       0.67      0.65      0.66       267
weighted avg       0.68      0.69      0.68       267
```

```
results['GB'] = gbc_score
```

```
sklearn.ensemble.GradientBoostingClassifier(*, loss='log_loss',
                learning_rate=0.1, n_estimators=100, subsample=1.0,
                criterion='friedman_mse', min_samples_split=2,
                min_samples_leaf=1, min_weight_fraction_leaf=0.0,
                max_depth=3, min_impurity_decrease=0.0, init=None,
                random_state=None, max_features=None, verbose=0,
                max_leaf_nodes=None, warm_start=False,
                validation_fraction=0.1, n_iter_no_change=None,
                tol=0.0001, ccp_alpha=0.0)
```



Boosting involves adding ensemble members sequentially to correct the predictions made by previous models, producing a weighted average of the predictions.

Ensemble Learning Models: Boosting

XGBoost

```
xgb_model = XGBClassifier(max_depth = 1, objective = 'reg:squarederror')
```

```
xgb_model.fit(X_train, y_train)
```

```
xgb_score = xgb_model.score(X_test, y_test)
```

```
print("Accuracy: %.2f%%" % (xgb_score * 100))
```

```
Accuracy: 67.42%
```

```
xgb_predictions = xgb_model.predict(X_test)
```

```
print(classification_report(y_test, xgb_predictions))
```

```
              precision    recall  f1-score   support

           0       0.69      0.84      0.76       162
           1       0.63      0.42      0.50       105

    accuracy                           0.67       267
   macro avg       0.66      0.63      0.63       267
weighted avg       0.67      0.67      0.66       267
```

```
results['XGB'] = xgb_score
```

```
xgboost.sklearn.XGBClassifier(base_score=None, booster=None,
              callbacks=None, colsample_bylevel=None,
              colsample_bynode=None, colsample_bytree=None,
              device=None, early_stopping_rounds=None,
              enable_categorical=False, eval_metric=None,
              feature_types=None, gamma=None,
              grow_policy=None, importance_type=None,
              interaction_constraints=None,
              learning_rate=None, max_bin=None,
              max_cat_threshold=None, max_cat_to_onehot=None,
              max_delta_step=None, max_depth=None,
              max_leaves=None, min_child_weight=None,
              missing=nan, monotone_constraints=None,
              multi_strategy=None, n_estimators=None,
              n_jobs=None, num_parallel_tree=None,
              random_state=None, ...)
```

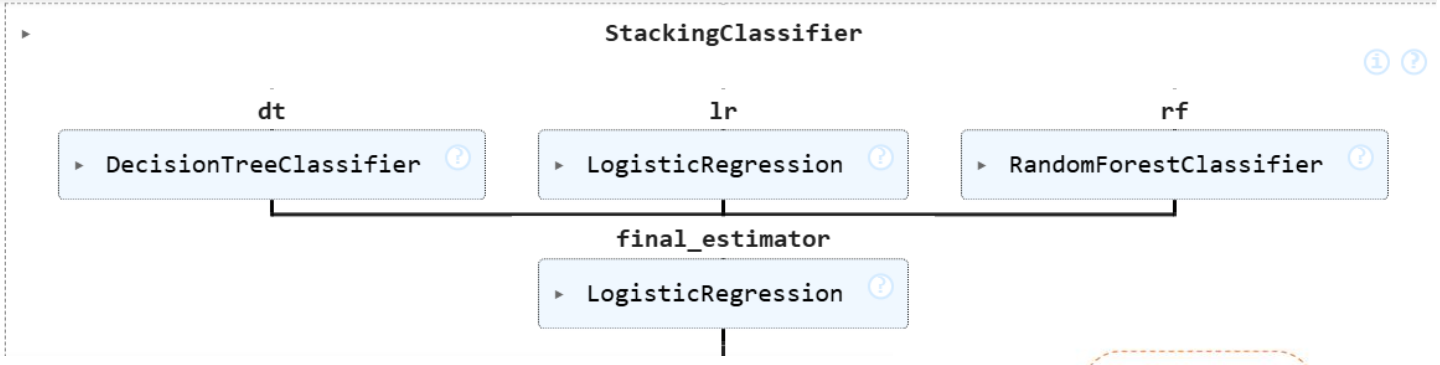
Ensemble Learning Models: Stacking

Stacking Classifier

```
estimators = [("dt", dt_model), ("lr", lr_model), ("rf", rf_model)]
```

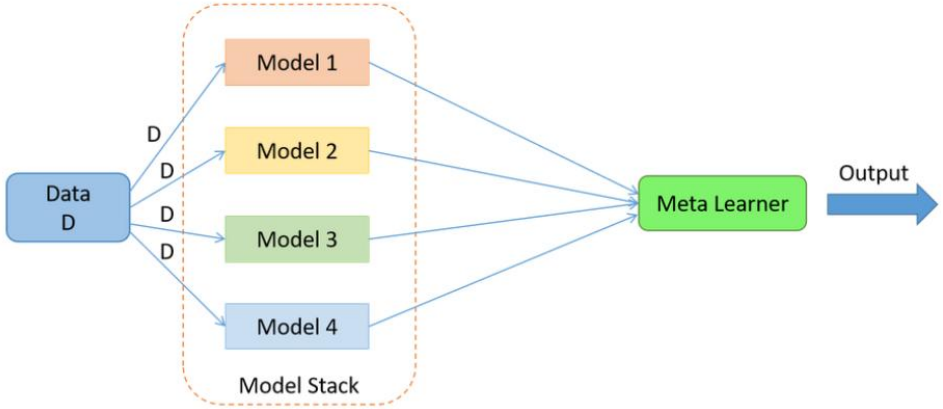
```
st_model = StackingClassifier(estimators = estimators,
                              final_estimator = LogisticRegression())
```

```
st_model.fit(X_train, y_train)
```




```
sklearn.ensemble.StackingClassifier(estimators,
                                    final_estimator=None, *, cv=None,
                                    stack_method='auto', n_jobs=None,
                                    passthrough=False, verbose=0)
```

Stacking involves fitting different types of model to the same dataset and using one model to learn the most effective way to combine the predictions.



Ensemble Learning Models: Stacking


```
st_score = st_model.score(X_test, y_test)
```

```
print("Accuracy: %.2f%%" % (st_score*100))
```

```
Accuracy: 67.79%
```

```
st_predictions = st_model.predict(X_test)
```

```
print(classification_report(y_test, st_predictions))
```

```
              precision    recall  f1-score   support

           0       0.70      0.83      0.76       162
           1       0.63      0.44      0.52       105

    accuracy                           0.68       267
   macro avg       0.66      0.64      0.64       267
weighted avg       0.67      0.68      0.66       267
```

```
results['Stack']= st_score
```

Model Accuracy Comparison

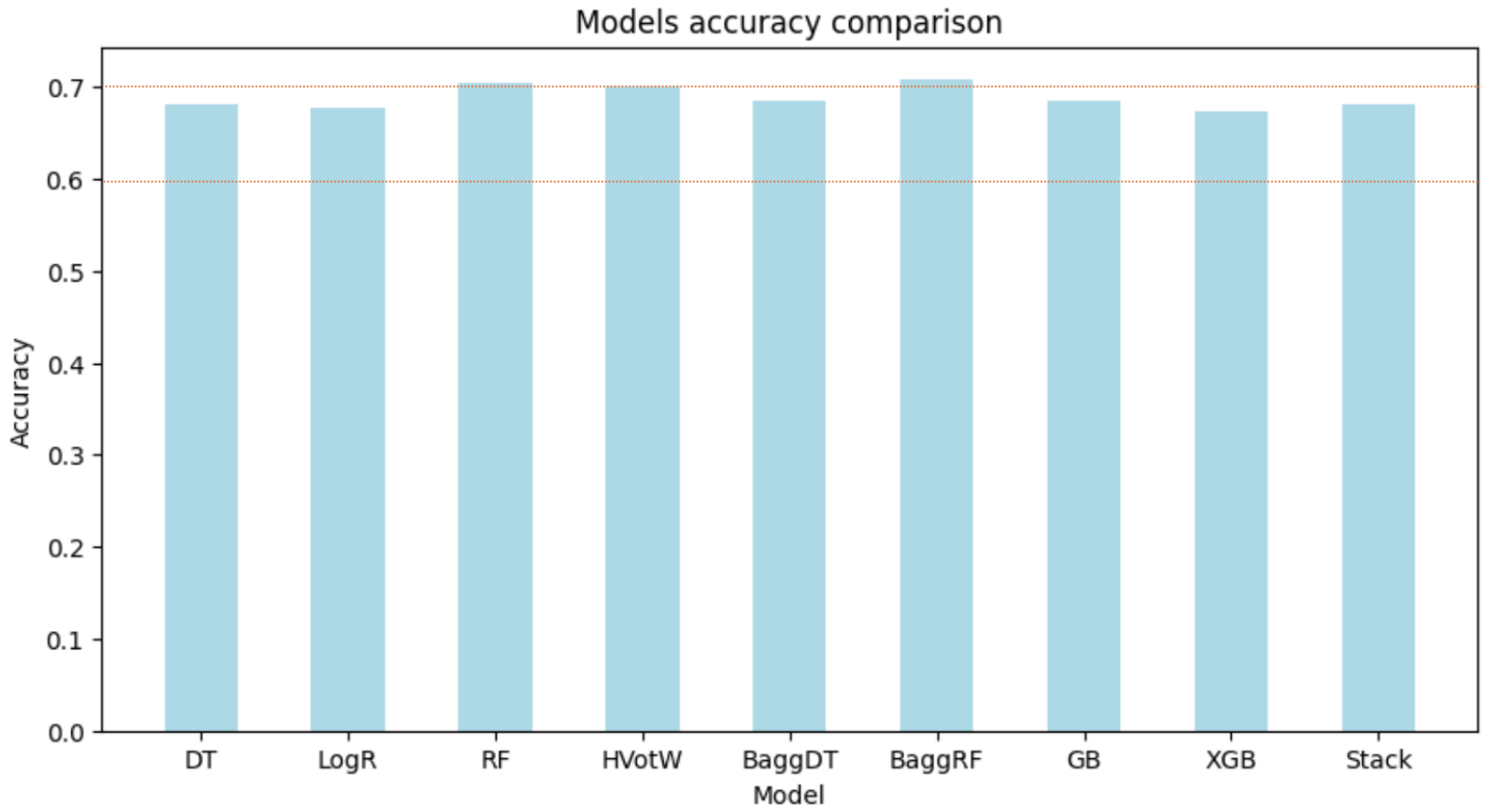
Create a bar chart showing the accuracy of each model:

```
fig = plt.figure(figsize = (10, 5))

mod = list(results.keys())
acc =list(results.values())

plt.bar(mod, acc, color ='lightblue',
        width = 0.5)

plt.xlabel("Model")
plt.ylabel("Accuracy")
plt.title("Models accuracy comparison")
plt.show()
```



Model Accuracy Comparison

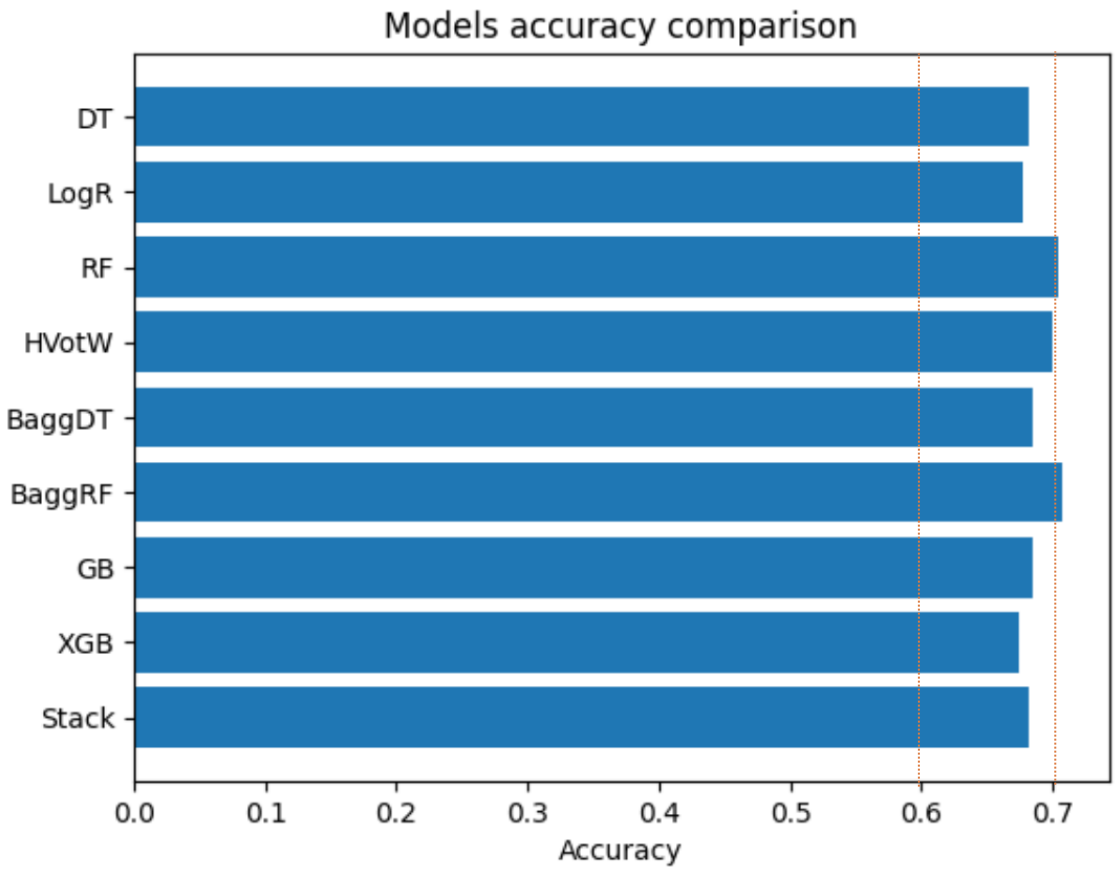
We can visualize this using horizontal bars:

```
fig, ax = plt.subplots()

y_values = np.arange(len(mod))

ax.barh(y_values, acc, align='center')
ax.set_yticks(y_values, labels = mod)
ax.invert_yaxis()
ax.set_xlabel('Accuracy')
ax.set_title('Models accuracy comparison')

plt.show()
```



Model Accuracy Comparison

Alternatively, we can print the results:

```
print("Models accuracy comparison")
for key, value in results.items():
    print("%s \t %.2f" % (key, value))
```

```
Models accuracy comparison
DT       0.68
LogR     0.68
RF       0.70
HVotW    0.70
BaggDT   0.69
BaggRF   0.71
GB       0.69
XGB      0.67
Stack    0.68
```

Which model performed better?
Which models are worth tuning?

Model Accuracy Comparison

Alternatively, we can print the results:

```
print("Models accuracy comparison")
for key, value in results.items():
    print("%s \t %.2f" % (key, value))
```

```
Models accuracy comparison
DT       0.68
LogR     0.68
RF       0.70
HVotW    0.70
BaggDT   0.69
BaggRF   0.71
GB       0.69
XGB      0.67
Stack    0.68
```

Which model performed better?
Which models are worth tuning?

Hands On